

Proposal for CNIC-based Health Card App: Front-End & Back-End Design

1 Project overview

Pakistan's health-care system is fragmented and relies heavily on paper-based prescribing. The government is already piloting a **One Patient One ID** program that uses the **CNIC number** as a permanent medical record number ¹. Building a centralised, card-based health information system that works on smartphones and web browsers could improve continuity of care, support telemedicine and reduce duplication. This proposal details the functional and technical requirements for such a system – including separate apps for patients and doctors, an admin web dashboard, CNIC-based authentication, a QR-coded health card, and real-time integration with electronic health record (EHR) systems using the HL7 FHIR standard.

2 Objectives

1. **Universal health record** – create a secure, longitudinal medical history linked to the patient's CNIC that can be accessed by authorised doctors anywhere in Pakistan.
2. **Ease of use** – design patient and doctor apps for Android and iOS with localised UI (English and Urdu) and offline caching to accommodate users without constant connectivity.
3. **Real-time update and viewing** – allow doctors to view a patient's history and add diagnoses, symptoms, medications and dosages that immediately appear on the patient side.
4. **Compliance** – adhere to Pakistan's emerging **Personal Data Protection Bill** (PDPB 2023) which regulates handling of sensitive data (including CNIC numbers and health information) ² and requires breach notification and data residency ³. Follow international best practices (HIPAA/HITECH) for encryption, audit logging and authentication ⁴.

3 Regulatory & ethical considerations

- **Sensitive data classification:** Under the PDPB, health data, CNIC numbers and biometrics are classified as *sensitive personal data* ². The platform must collect the minimum necessary data, obtain explicit consent and allow patients to request deletion (right to erasure within 14 days) ⁵.
- **Data residency:** Critical personal data must be processed on servers located in Pakistan ³. Cloud hosting should therefore use local data centres or a hybrid on-premises/cloud setup.
- **Security & risk management:** Conduct a risk assessment and implement technical safeguards such as encryption, multi-factor authentication (MFA), role-based access control (RBAC) and audit logging ⁴. A technical blueprint for secure patient portals recommends MFA with passwords and OTPs/ biometrics, RBAC to restrict data by user type, session timeouts, AES-256 encryption at rest and TLS in transit, and comprehensive audit logs ⁶.
- **Consent management:** Use a standard consent form for data sharing and ensure versioned consent is recorded, as recommended in telemedicine system design ⁷.

4 Functional requirements

4.1 User roles

Role	Capabilities
Patient	Register using CNIC, phone and password; receive OTP for verification; view personal demographic data; scan their health card QR code to add themselves; view chronological health timeline (diagnoses, symptoms, medications, lab results); book appointments and receive reminders; securely message doctors; request prescription refills; update contact information; choose privacy settings (e.g., hide sensitive visits).
Doctor	Register with PMDC number and CNIC after verification by the admin; search or scan patient health cards; view patient summaries; add or edit diagnoses, symptoms, medications and dosages (FHIR <code>Observation</code> , <code>Condition</code> and <code>MedicationRequest</code> resources); upload lab results or attach documents; add notes; communicate with patients via secure messaging; schedule follow-up appointments; access analytics on their panel.
Admin (Health authority)	Manage doctor credentialing; approve or deny account requests; assign roles; manage health card issuance and QR generation; configure EHR integrations; run reports; manage consent policies and system settings.

4.2 Core user journeys

1. Account creation & login

2. Patient provides CNIC, phone number and password; the system sends an OTP via SMS to verify identity; once verified, a QR-coded digital health card is generated and optionally printed.
3. Doctors provide PMDC number, CNIC and credentials; admin reviews and approves; doctors then set a password and complete MFA setup.
4. On login, users enter CNIC and password; a one-time code is sent; user must complete MFA for high-risk actions (e.g., modifying data).

5. Viewing health history

6. Patient opens the app and sees a chronological timeline of visits, diagnoses and medications. Each entry links to FHIR resources.
7. Doctor scans the patient's QR code or searches by CNIC; the app requests the FHIR server for the patient's `Patient` resource and associated `Encounter`, `Condition`, `Observation` and `MedicationStatement` resources; the doctor views the record and adds notes.

8. Entering clinical data

9. After a consultation, the doctor selects the patient and uses forms to record symptoms, diagnosis, treatment plans and medication dosages. Each form maps to FHIR resources and is validated. The update is sent via API to the FHIR server; through FHIR **subscriptions** the patient app receives a notification and updates its local copy ⁸ .

10. Messaging & follow-ups

11. Secure messaging is available to ask non-urgent questions, refill prescriptions or provide lifestyle advice. According to healthit.gov, secure messaging improves care coordination and allows patients to communicate at their convenience ⁹ .

12. Patients can schedule appointments and receive reminders; doctors can manage their appointment calendar.

4.3 Additional modules

- **Appointment scheduling & reminders:** As noted in patient portal best practices, portals should allow patients to schedule, change or cancel appointments and receive reminders via SMS or push notifications ¹⁰ .
- **Medication refill & adherence:** Patients can request refills and receive alerts when it's time to renew medication ¹¹ .
- **Insurance & billing integration:** While not required initially, support for displaying insurance coverage and claims can be added later ¹² .
- **Telehealth integration:** Provide video consultations using secure WebRTC; integrate with existing telemedicine platforms (e.g., Sehat Kahani) via APIs. Ensure that notifications do not reveal sensitive details (e.g., do not mention psychiatry visits) ¹³ .
- **Reporting & analytics:** Admins and doctors can run reports on patient demographics, disease trends and medication adherence for decision support.

5 Technical architecture

5.1 Overall architecture

Use a modular, **FHIR-first** architecture so that all clinical data are stored and exchanged as FHIR resources. A FHIR-first design builds the application on FHIR API and data models from the start ¹⁴ , enabling structured, modular data and easy integration with other systems ¹⁵ . Key components include:

1. **Mobile apps (Android/iOS)**
2. **Patient App:** Cross-platform (Flutter or React Native) for iOS and Android. UI supports English/Urdu and right-to-left text. Offline caching of the patient's record for areas with poor connectivity; when offline, updates are queued and synced once connected.
3. **Doctor App:** Designed for physicians; supports complex workflows (e.g., capturing detailed clinical data, prescribing) and quick scanning of QR codes. Design separate patient and provider apps because their UI requirements differ dramatically ¹⁶ .

4. **Admin Web Portal** – built using a modern JS framework (React or Angular). Provides dashboards for managing users, generating health cards, monitoring usage and configuring integrations.
5. **API Gateway** – central entry point for all client requests. Provides rate limiting, routing, authentication and load balancing.
6. **Identity and Consent Service** – handles CNIC-based authentication, MFA (password + OTP/ biometric), session management, RBAC and consent management. Based on OpenID Connect/ SMART on FHIR for standardised authentication and authorisation ⁷ .
7. **FHIR Server** – hosts all medical data as FHIR resources. Use an open-source server like **HAPI FHIR**, which is widely used, interoperable and can be deployed via Docker ¹⁷ . HAPI FHIR uses RESTful APIs and supports both JSON and XML ¹⁸ . Deploy at least two instances for redundancy and host them on local data centres to comply with data residency requirements.
8. **Application Services** – microservices built with Node.js/Express (or Spring Boot) that implement domain logic: patient management, doctor management, appointment scheduling, messaging, notifications, and reporting. Node.js provides a unified development environment and is lightweight; Express simplifies routing and HTTP operations ¹⁹ .
9. **Integration Layer** – connectors for:
10. **NADRA CNIC verification**: call NADRA API to verify CNIC and fetch demographic data ²⁰ .
11. **Laboratory Information Systems (LIS), Radiology (DICOM) and Pharmacy**: integrate via FHIR `Observation` , `DiagnosticReport` and `MedicationDispense` resources.
12. **Telemedicine platforms**: integrate with existing telehealth providers (e.g., Sehat Kahani) to allow video calls.
13. **Messaging & Notification Service** – provides secure message threads between doctors and patients, and sends push/SMS notifications. It logs all interactions in the FHIR server (e.g., `Communication` resource) and ensures messages are encrypted end-to-end.
14. **Analytics & Audit Service** – collects system logs, monitors access patterns and generates audit trails. It provides dashboards for administrators and ensures all access to patient data is logged for compliance (who viewed/modified which resource and when).

5.2 Data model

- Each **Patient** is identified by a unique CNIC; the FHIR `Patient.identifier` uses `system="https://nadra.gov.pk/cnic"` with the CNIC number. Additional identifiers (e.g., insurance numbers) can be added later.
- **Encounters** store visits; `Encounter.subject` references the patient and `Encounter.participant` references the doctor.
- **Conditions** describe diagnoses; `Condition.code` uses SNOMED CT or ICD-10 codes for standardisation ²¹ .

- **Observations** capture vitals, lab results and symptoms; `Observation.valueQuantity` records measurements.
- **MedicationRequest** stores prescribed medicines and dosages; `Medication` resources describe the drug (brand and generic names). When a doctor updates medications, a new `MedicationRequest` is created and the patient app is notified via FHIR subscriptions ⁸.
- **Communication** resources represent secure messages.
- **Provenance** resources track who created/updated each resource and when, providing an audit trail.

5.3 Security & compliance

1. **Authentication & MFA** – Combine password authentication with **one-time passwords (OTPs)** via SMS; optionally add biometric authentication (e.g., fingerprint or facial recognition) on devices that support it ⁶.
2. **Role-Based Access Control (RBAC)** – assign roles (patient, doctor, admin). Doctors can only access records of patients they are treating. Patients can only view their own record. Admins cannot view clinical details but can manage metadata. The DI Solutions blueprint emphasises RBAC to restrict data visibility based on user type ⁶.
3. **Encryption** – Use **TLS/HTTPS** for all data in transit and **AES-256** encryption for data at rest ⁶. Protect cryptographic keys in a hardware security module (HSM).
4. **Audit logging & monitoring** – Log user logins, failed attempts, data accessed/modified and new device access ⁶. Provide monitoring dashboards for suspicious activities.
5. **Session management** – Implement session timeouts and automatic logout after periods of inactivity ⁶.
6. **Risk assessment & compliance** – Conduct regular security audits and penetration tests; evaluate vulnerabilities; keep software dependencies updated. Use frameworks that support HIPAA and PDPB compliance. DI Solutions recommends risk assessments, choosing compliant technology stacks (e.g., PostgreSQL with built-in encryption), implementing technical safeguards, designing accessible UX and partnering with trusted vendors ²².
7. **Consent & privacy** – Provide transparency on data use; log patient consents; allow patients to revoke access. Store only the minimum necessary data; never reveal details in push notifications (e.g., appointment type) ¹³.

6 Front-end design considerations

1. **Cross-platform development** – Use **Flutter** or **React Native** so that one codebase serves Android and iOS, reducing development cost. Use **Localization** packages to support English and Urdu; allow users to switch language at runtime.
2. **Accessibility** – Follow WCAG 2.1 Level AA guidelines for contrast, font size and voice-over support. For low-literacy users, provide icons and audio prompts. For left-to-right (Urdu) support, ensure UI can flip direction.
3. **Offline support & data sync** – Implement a local database (e.g., SQLite or Realm) to store a read-only snapshot of the patient's record. Use FHIR **Subscription** mechanisms to synchronise data when connectivity returns ⁸.
4. **QR scanning & generation** – Integrate camera access to scan patient health cards. The QR code encodes the CNIC and possibly an opaque token; scanning triggers fetching the patient's record via the API. Use libraries like `react-native-camera` or `Flutter Barcode Scanner`.

5. **Forms & validation** – Create dynamic forms mapped to FHIR resources; for example, use dropdowns for ICD-10 codes, search for medications, and allow free-text notes. Validate fields on the client side; provide progress indicators when uploading data.
6. **Secure messaging** – Offer threaded conversations; allow attachments (lab reports, pictures); implement push notifications via Firebase Cloud Messaging or Apple Push Notification Service; store messages in FHIR `Communication` resources.
7. **Notifications & reminders** – Provide configurable reminders for appointments and medication. For privacy, notifications should be generic (e.g., “You have a new message in the Health Card app”) ¹³.

7 Back-end implementation details

1. **Microservices architecture** – Organise services by domain (e.g., user service, FHIR service proxy, appointment service, messaging service). Each service has its own database or uses FHIR as the source of truth. Use Docker and Kubernetes for container orchestration.
2. **API design** – Provide a RESTful API conforming to FHIR where possible. Additional endpoints can wrap FHIR operations (e.g., `POST /patients/{id}/encounters` creates an encounter). Implement GraphQL if flexible queries are needed. Use asynchronous messaging (e.g., RabbitMQ or Apache Kafka) for events like new encounter creation to trigger notifications.
3. **Database** – Use **PostgreSQL** with Transparent Data Encryption for non-FHIR tables (user accounts, audit logs). FHIR data resides in the HAPI FHIR database (which can be configured to use PostgreSQL or MySQL). Use **Elasticsearch** for full-text search across patient records.
4. **Caching & performance** – Use Redis for session storage and caching frequently accessed data (e.g., dictionary lists). Implement rate limiting at the API gateway.
5. **CNIC verification module** – Integrate NADRA API with strong encryption. The module sends CNIC to NADRA, optionally matches biometrics, and receives demographic data to populate the patient record ²⁰. Caching results avoids repeated calls.
6. **Identity and access management** – Use an OpenID Connect provider (e.g., Keycloak or Auth0) to handle authentication, MFA and RBAC. Configure identity claims to include user role and organisation.
7. **Monitoring & logging** – Implement centralised logging (e.g., with ELK stack) and metrics collection (Prometheus + Grafana). Set up alerts for unusual access patterns and service failures.

8 Development roadmap and phases

1. **Discovery & planning (1–2 months)**
 2. Conduct stakeholder interviews (government, NADRA, hospitals, telemedicine providers, patient advocacy groups).
 3. Define minimum viable product (MVP) scope and success metrics (e.g., number of patients enrolled, number of consultations recorded).
 4. Draft detailed user stories and data flow diagrams.
5. **Architecture design & prototyping (2 months)**
 6. Select technology stack (Flutter vs React Native; Node.js vs Spring Boot; database choices).

7. Deploy HAPI FHIR server in staging environment; set up CNIC verification sandbox.
8. Build wireframes and interactive prototypes for patient and doctor apps; conduct usability testing with sample users.

9. Core development (4–6 months)

10. **Back-end:** Implement identity service, FHIR proxy service, appointment service, messaging service and admin portal. Integrate NADRA API. Set up RBAC.
11. **Front-end:** Develop patient app features (registration, health timeline, QR scanning, messaging, appointments) and doctor app features (patient search, add notes/medications, view timeline). Implement multi-language support.
12. **Security & compliance:** Implement encryption, MFA, audit logging, data retention policies. Create standard consent forms and privacy policies.

13. Integration & interoperability (1–2 months)

14. Connect to pilot hospitals' EHR systems via FHIR; map FHIR resources to local codes (SNOMED, ICD). Test retrieving lab results and medication lists.
15. Integrate telehealth video module and ensure notifications align with privacy guidelines ¹³.

16. Testing & quality assurance (2 months)

17. Conduct unit, integration and performance tests. Perform security penetration testing and threat modelling. Validate compliance with PDPB and HIPAA guidelines.
18. Pilot with a small number of clinics; gather feedback; adjust UI/UX.

19. Pilot & deployment (3 months)

20. Onboard selected healthcare providers and issue physical/digital health cards to patients. Provide training and support.
21. Monitor usage, performance and user satisfaction; collect feedback; fix bugs.
22. Prepare for scale-up to other regions; plan for offline kiosks in rural areas for enrollment.

23. Ongoing maintenance & enhancements

24. Implement continuous integration/continuous deployment (CI/CD) pipelines; release regular updates.
25. Add additional features such as insurance integration, analytics dashboards, integration with wearable devices and AI-powered decision support.

26. Conduct periodic security and compliance audits and update to future PDPB regulations.

9 Conclusion

A CNIC-based health card system that uses FHIR-first design and robust security practices can transform healthcare delivery in Pakistan. By providing separate patient and doctor apps, integrating with NADRA for identity verification, and complying with the PDPB and HIPAA guidelines, the platform ensures secure, continuous, and patient-centred care. The blueprint above outlines the functional requirements, technical architecture, data model, security measures and development roadmap necessary for a software agency to start building a scalable, secure and user-friendly solution. Adhering to these guidelines will align the project with Pakistan's **One Patient One ID** initiative ¹ and global interoperability standards, while ensuring sustainability and adoption across diverse populations.

¹ Pakistan launches one patient, one ID to transform healthcare access | Biometric Update
<https://www.biometricupdate.com/202504/pakistan-launches-one-patient-one-id-to-transform-healthcare-access>

² ³ ⁵ Navigating the New Frontier of Data Privacy in Pakistan - ABS & Co
<https://absco.pk/insights/navigating-the-new-frontier-of-data-privacy-in-pakistan/>

⁴ Develop a privacy and security telehealth strategy | Telehealth.HHS.gov
<https://telehealth.hhs.gov/providers/best-practice-guides/privacy-and-security-telehealth/develop-privacy-and-security>

⁶ ²² DI Solutions
<https://disolutions.net/blog/technical-blueprint-building-secure-patient-portals>

⁷ ¹³ ¹⁶ Telemedicine Platform Architecture: The End-to-End Map
<https://www.forasoft.com/learn/telemedicine/articles-telemedicine/anatomy-of-a-telemedicine-platform>

⁸ ¹⁴ ¹⁵ FHIR-First Design for Digital Health Apps: Security, Performance, and Sync
<https://www.capminds.com/blog/fhir-first-design-for-digital-health-apps-security-performance-and-sync/>

⁹ nlc-secure-messaging-fact-sheet.pdf
<https://healthit.gov/wp-content/uploads/2026/01/nlc-secure-messaging-fact-sheet.pdf>

¹⁰ ¹¹ ¹² Build A Patient Portal App Like MyChart In 2025
<https://digitrends.co/blog/build-a-patient-portal-app-like-mychart/>

¹⁷ ¹⁸ ¹⁹ Building an Electronic Medical Record System Exchanged in FHIR Format and Its Visual Presentation - PMC
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10486699/>

²⁰ Linking CNIC with Medical Records: NADRA Compliance for HMIS
<https://www.itacksolutions.com/nadra-compliance/>

²¹ Healthcare EHR Interoperability in 2025: How to Choose the Right Solution
<https://www.capminds.com/blog/healthcare-ehr-interoperability-in-2024-how-to-choose-the-right-solution/>